

8.4 STATE CHART DIAGRAM

A state chart diagram is used in computer science and related fields to describe the behavior of systems. It is a type of UML diagram that illustrates the various states of an object and the transitions between these states in response to events.

8.5 USES OF STATE CHART DIAGRAM

- i. State chart diagram emphasizes the event-ordered behavior of an object.
- ii. State diagrams are used to model the states, transitions, events, and activities of a system component, such as a class, interface, or collaboration.
- iv. Demonstrating the response of a system to different types of stimuli.
- v. State chart diagrams are also used for forward and reverse engineering of a system.

8.6 BASIC ELEMENTS OF STATE CHART DIAGRAM

- Transition: a transition refers to a change or shift from one state, subject, place, or situation to another.
- State Actions: are actions that occur when an object transitions from one state to another.
- Entry actions/point: These actions are defined for the target state and are invoked when the object enters the state.
- Exit actions/point: These actions are defined for the source state. Exit actions are the opposite of entry actions
- History States: A history state is used to remember the last sub state that was active.
- Concurrent Regions: allow multiple activities to occur simultaneously within the context of the enclosing object.

8.7 PROCESS OF MODELING WITH STATE CHART DIAGRAM

Here is a process of modeling using a state chart diagram:

1. Identify States
2. Define Transitions
3. Establish Initial and Final States
4. Draw the Diagram
5. Add Actions and Activities
6. Validate the Diagram

Example State Chart Diagram

Here's a simple example of a state chart diagram for an online order process:



Fig. 8 State chart diagram

In this diagram:

- States: `Order Placed`, `Order Packed`, and `Order Shipped`.
- Transitions: The order moves from `Order Placed` to `Order Packed` when payment is successful. It then moves to `Order Shipped`.
- Initial and Final States: The process starts at the initial state and ends at the final state.

9.0 INTERACTION DIAGRAM

An interaction diagram is a type of UML diagram that captures the interactive behavior of a system. It focuses on describing the flow of messages within a system, providing context for one or more lifelines within a system. Interaction diagrams can be used to represent the ordered sequences within a system and act as a means of visualizing real-time data via UML.

9.1 TYPES OF INTERACTION DIAGRAM

There are four types of interaction diagrams in UML, which are:

- i. Sequence diagram
- ii. Collaboration diagram
- iii. Timing diagram
- iv. Interaction overview diagram

10.0 SYSTEM MODEL CONVERSION

System model conversion refers to the process of converting a model from one representation to another.

10.1 IMPORTANCE OF MODEL CONVERSION

Model conversion is essential for ensuring the following

- i. Compatibility
- ii. Optimization
- iii. Reusability
- iv. Scalability
- v. Interoperability
- vi. Portability
- vii. Customization

10.2 PROCESS OF CONVERTING UML DIAGRAMS INTO PROGRAM CODE

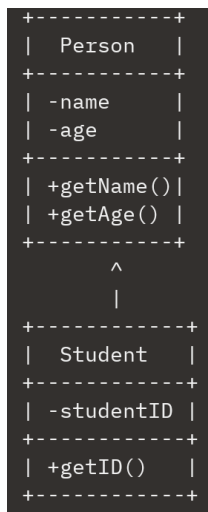
To convert UML diagrams into program code, you can use a code generator tool that allows you to convert your diagrams to source code in various programming languages. The code generator supports various programming languages, including Java, C#, Python, C++, and others.

The process of converting UML diagrams into program code involves the following steps:

- Create a UML diagram using a UML modeling tool.
- Use a code generator tool to generate code from the UML diagram. The code generator tool will convert the UML diagram into source code in the programming language of your choice.
- Refine the generated code to ensure that it meets the requirements of the system.
- Compile and test the code to ensure that it works as expected.

Example:

Assume a simple class diagram with a `Person` class and a `Student` subclass.



```
java Copy

public class Person {
    private String name;
    private int age;

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }
}

public class Student extends Person {
    private String studentID;

    public String getID() {
        return studentID;
    }
}
```

The code shows the implementation of the classes in Java. The 'Person' class has private attributes 'name' and 'age', and public methods 'getName()' and 'getAge()'. The 'Student' class extends 'Person' and has a private attribute 'studentID' and a public method 'getID()'.

10.3 PROCESS OF CONVERTING PROGRAM CODE INTO UML DIAGRAMS

The process of converting program code into UML diagrams can be challenging, but there are tools and techniques available to help with this process.

Here are some steps that can be followed to convert program code into UML diagrams:

- Analyze the program code to understand the structure and behavior of the system.
- Identify the classes and objects within the code that are involved in the system.
- Create a class diagram that represents the classes and objects within the code and their relationships.
- Add attributes and methods to the class diagram to represent the behavior of the system.
- Validate the class diagram with stakeholders to ensure accuracy, completeness, and consistency.

Assignment

1. Explain the basic elements and notation of use case diagram
2. Explain the following implementation diagram
 - i. Sequence diagram
 - ii. Collaboration diagramUse diagram to support your answer